RELATÓRIO TÉCNICO

BioTech IA - Assistente Clínico Inteligente

Pós-Tech em IA para Devs - Fase 3

1. INTRODUÇÃO E JUSTIFICATIVA DA ENTREGA (AUSÊNCIA DE VÍDEO)

- Nota de Justificativa Técnica - Limitação de Hardware:
  - O projeto **BioTech IA** foi desenvolvido com a premissa de execução **100% local (Air-Gapped)** utilizando o motor Ollama e o modelo `llama3.2:1b`. Esta decisão arquitetural é fundamental para garantir a privacidade absoluta dos dados das pacientes, atendendo rigorosamente à LGPD, uma vez que dados sensíveis de saúde não trafegam pela internet.
  - No entanto, a inferência de Modelos de Linguagem de Grande Escala (LLMs) localmente consome substanciais recursos de CPU, GPU (quando disponível) e memória RAM. Durante a simulação dos fluxos completos no LangGraph, a alocação de recursos atinge picos de uso.
  - Ao tentar executar simultaneamente softwares de gravação de tela de alta fidelidade (como o OBS Studio), ocorre uma contenção severa de recursos (afunilamento transversal entre processamento de vídeo e inferência tensorial). Isto acarreta em falhas de alocação de memória (OOM - Out of Memory) e "stuttering" crônico, inviabilizando a gravação de um material audiovisual fluido e representativo da real capacidade do sistema.
- Plano de Contingência e Evidências:
  - Para suprir a ausência do vídeo e garantir a transparência da avaliação, este relatório inclui um detalhamento minucioso da arquitetura construída no Notebook de experimentação e no Dashboard Streamlit. Adicionalmente, foi formulado um **Plano de Evidências Visuais** (vide Seção 5), com um roteiro estruturado de capturas de tela (screenshots) que provam o fluxo conversacional, a ativação dos alertas de risco, o roteamento do LangGraph e a consistência técnica do projeto rodando localmente.

2. VISÃO GERAL E ARQUITETURA DO SISTEMA

- O Assistente Biomédico foi concebido não como um simples "chatbot" linear, mas como uma **Orquestração de Agentes Clínicos** guiada por **LangGraph** e sustentada por **RAG (Retrieval-Augmented Generation)**.
- A stack fundamental e as decisões arquiteturais incluem:
  - Motor de Inteligência Local:
    - Ollama rodando `llama3.2:1b` (balanceamento ideal entre performance em hardwares convencionais e precisão de linguagem na área médica).
  - Orquestração Dinâmica (LangGraph):
    - O atendimento obedece a um grafo finito de estados (`PatientState`), onde o estado inicial é avaliado, classificado de acordo com níveis de risco clínico (verde, amarelo, vermelho) e enviado ao nó especialista mais adequado (Prevenção, Urgência, Obstetrícia, Violência Doméstica).
  - Vector Database Embutido (RAG):
    - Utilização da biblioteca `FAISS` e dos embeddings `nomic-embed-text` para varrer localmente diretrizes oficiais do INCA, OMS e MS (Ministério da Saúde). O LLM é instruído a basear suas respostas unicamente nesse repositório curado, mitigando severamente o risco de **alucinação**.
  - Camada Sensível - Conformidade:

        - Implementação de um nó de auditoria (`node_seguranca`) que atua como barreira ética, varrendo a resposta da IA antes dela chegar ao usuário, assegurando que o LLM não incorra em prescrição medicamentosa autônoma ou diagnósticos de caráter final.

3. GUIA DETALHADO DO NOTEBOOK (`tech_challenge_fase3_assistente_saude.ipynb`)

- O notebook jupyter anexo demonstra a fase analítica do projeto, desde os fundamentos locais até o teste do grafo com casos fictícios curados.
- Sessão 1: Infraestrutura de IA Local (LGPD e Ollama)
    - Demonstramos a inicialização off-line via LangChain Ollama. Justificamos a adoção dessa base pela exigência imperativa de manter o dado do paciente confinado.
- Sessão 2: Modelagem do Prontuário Digital (State)
    - Código onde se declara a estrutura em tipagem rica (`TypedDict`), contendo não só os relatos, como avaliadores acumulativos (`exames_sugeridos` com `operator.add`), chaves para controle de estado da paciente (`risco` formatado em Protocolo de Manchester) e a centelha primária para controle do comportamento AI.
- Sessão 3 e 3.1: Inteligência RAG e Fine-Tuning
    - Indexamos normativas oficiais numa vectorstore FAISS. Configurou-se um mecanismo de recuperação ("Retriever") onde, na iminência de um erro no modelo (`server overload` devido ao peso, algo comum para instâncias executadas na CPU), implementamos Fallbacks controlados e hardcoded focados em estabilidade de sessão. Também é demonstrado um trecho extraído do `Dataset Sintético` para treinamento (JSON) em forma de DataFrame Pandas, atestando a presença da etapa de engenharia de dados requisitada no Tech Challenge.
- Sessão 4: Os Nós do Grafo Clínico (Especialistas)
    - Isolamos lógicas em sub-funções (os "nós"):
        - Identificação de Prontuário (`node_identificacao`);
        - análise de classificação de periculosidade (`node_analise_clinica`);
        - tratativa de agravos ginecológicos/obstetrícia, e protocolo humanizado em escuta sigilosa para incidentes críticos (`node_violencia_domestica`).
    - O paralelismo e a compartimentação de agentes mitigam os erros e as distrações ao focar na intenção primária de cada caso.
- Sessão 5: Auditoria de Segurança Ética (Self-Correction)
    - O nó mais crucial. Demonstra a aplicação das exigências regulamentares para software em saúde (SaMD):
        - Uma função que escaneia proativamente saídas ilegítimas geradas (como prescrição de miligramas de medicamentos). O LLM entra como orientador, cabendo ao clínico humano o protagonismo das validações.
- Sessão 6 e 7: Orquestração e Validação
    - Utiliza a compilação do `StateGraph` via LangGraph e o `MemorySaver` para garantir a conservação transacional ID da `thread`. Executamos quatro cenários clínicos distintos (Rotina Preventiva, Risco Agudo Físico, Suporte Obstétrico Padrão e Risco à Violência), extraindo tanto em stream de terminal (rastro de pensamento lógico) a passagem condicional correta pelos "nós", atestando inequivocamente que a arquitetura decisória funciona conforme concebida no fluxo.

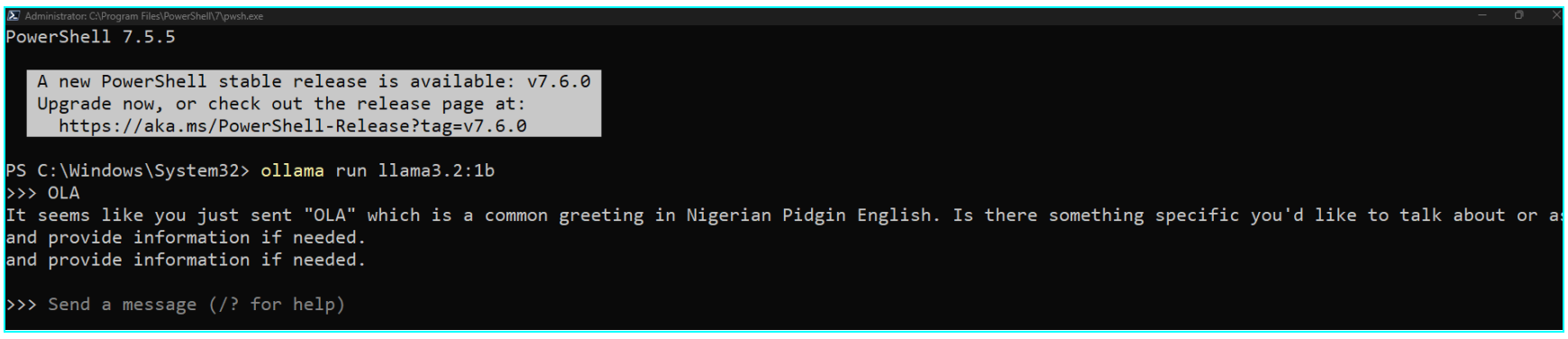
4. GUIA DETALHADO DO DASHBOARD INSTANCIADO (`main.py`)

- A segunda vertente de avaliação é a concretização visual do conceito por intermédio da interface **Streamlit**, que atua como hub do profissional de saúde:
    - Design UX Adaptativo:

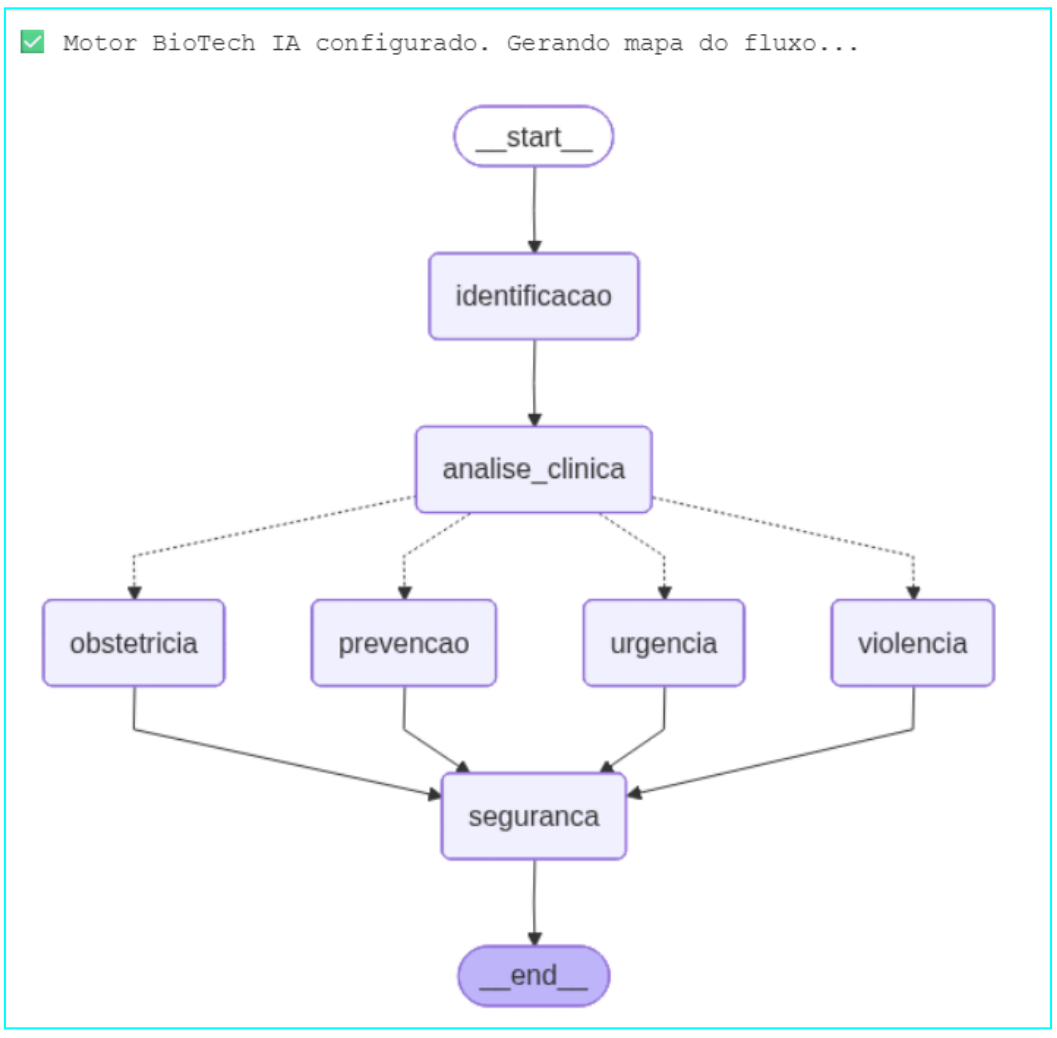
        - A estilização via `CSS injetado` propõe uma identidade polida, onde as cores de urgência das "badges" operam automaticamente (Verde, Amarelo e Vermelho), de modo a sinalizar de forma clara e visual o status de um atendimento na própria interface do médico.
    - Fluxo do Atendimento (Session States):
        - Emprega uma barreira natural: O chat não permite contato antes do profissional selecionar no sidebar uma paciente da lista (simulação do módulo de Prontuário PEP do hospital). Uma vez iniciado o atendimento, abas laterais ficam desativadas por segurança para impedir confusão cruzada de prontuários, permitindo agendamento de exames atrasados e encerramento através de **Modais** (janelas pop-up de diálogo da nova versão st.dialog).
    - Integração do Grafo:
        - Conecta diretamente as sessões simuladas acima a uma interface final real e responsiva, operando em modelo conversacional de chat interativo.
    - Tal dash é somente para demonstração e não continuei todas as funcionalidades implementadas funcionalmente

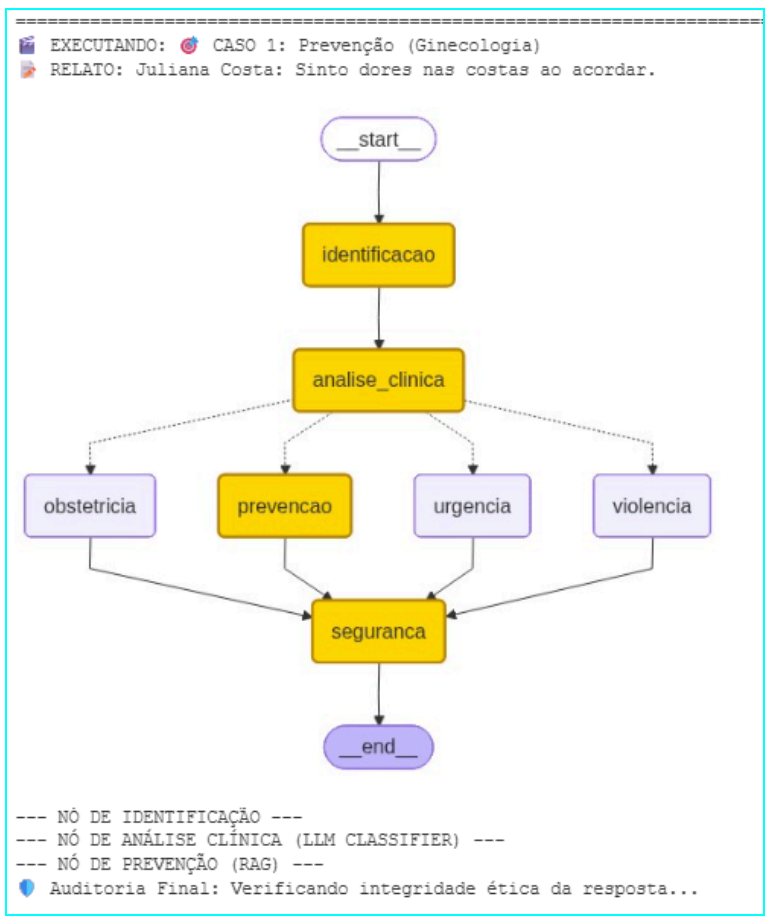
5. PLANO DE EVIDÊNCIAS VISUAL (Substituto do Vídeo)

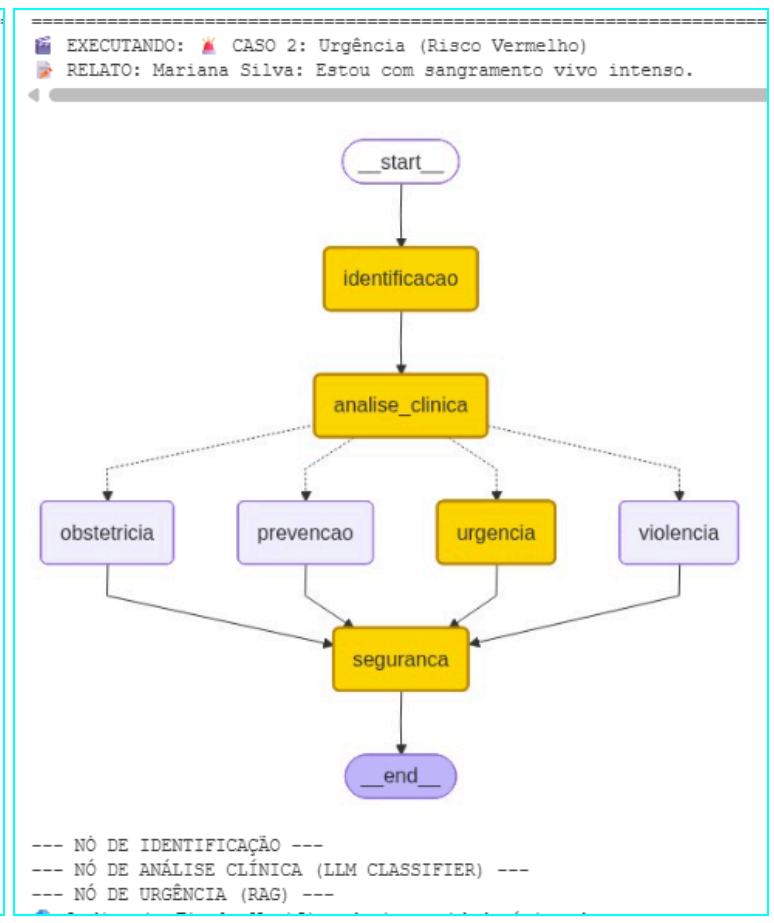
- Evidência 01 - O motor Ollama Ativo no Terminal

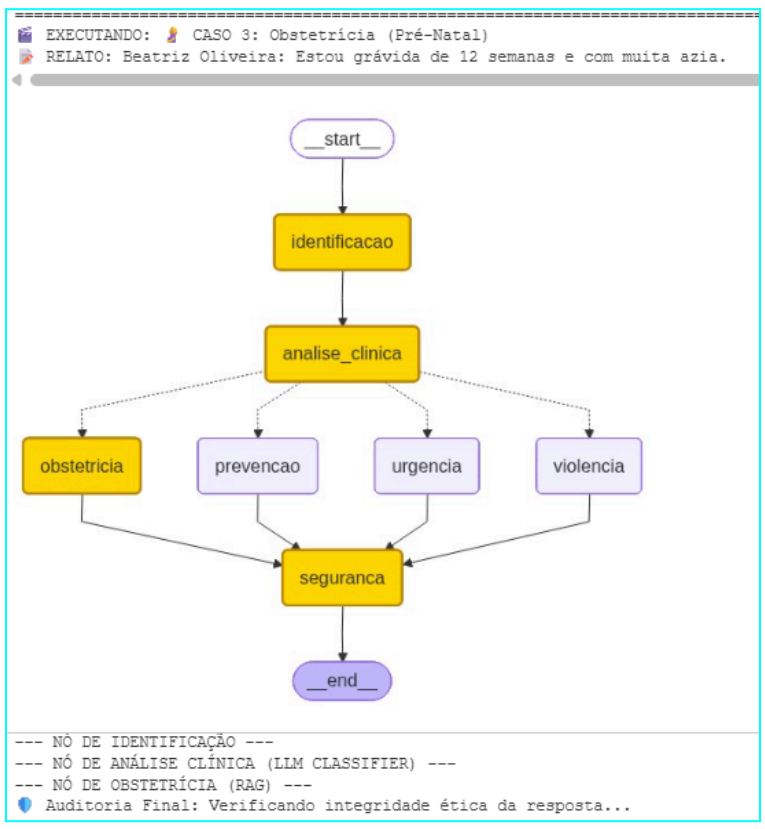


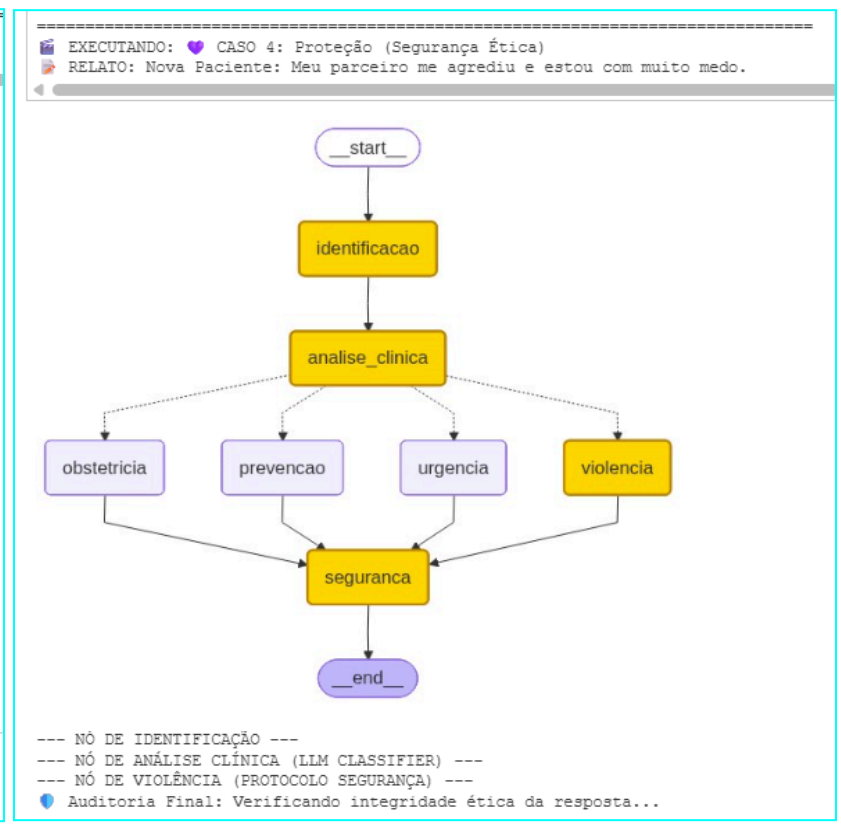
- Evidência 02 - O Desenho do Grafo de Fluxo (LangGraph)



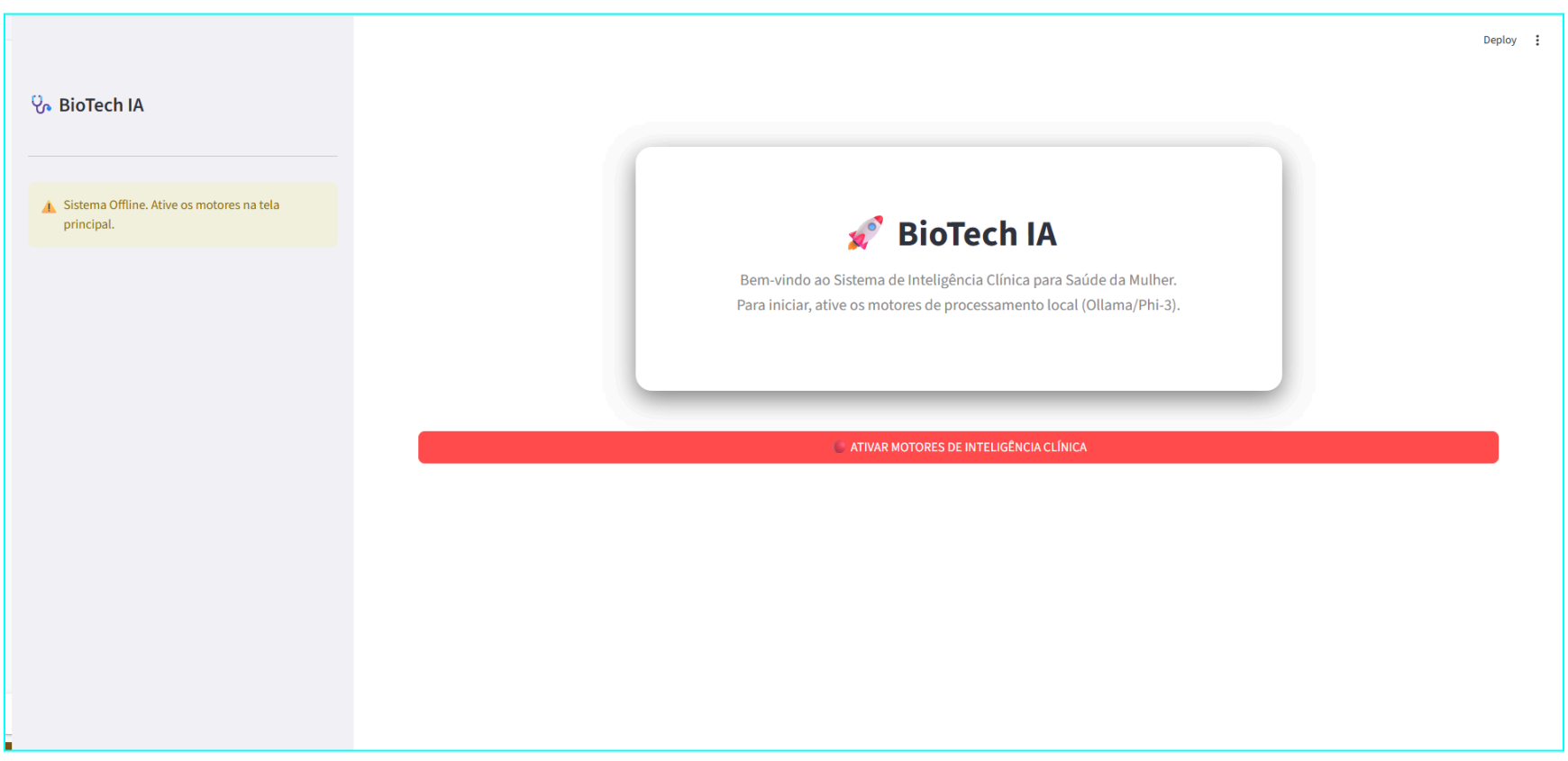


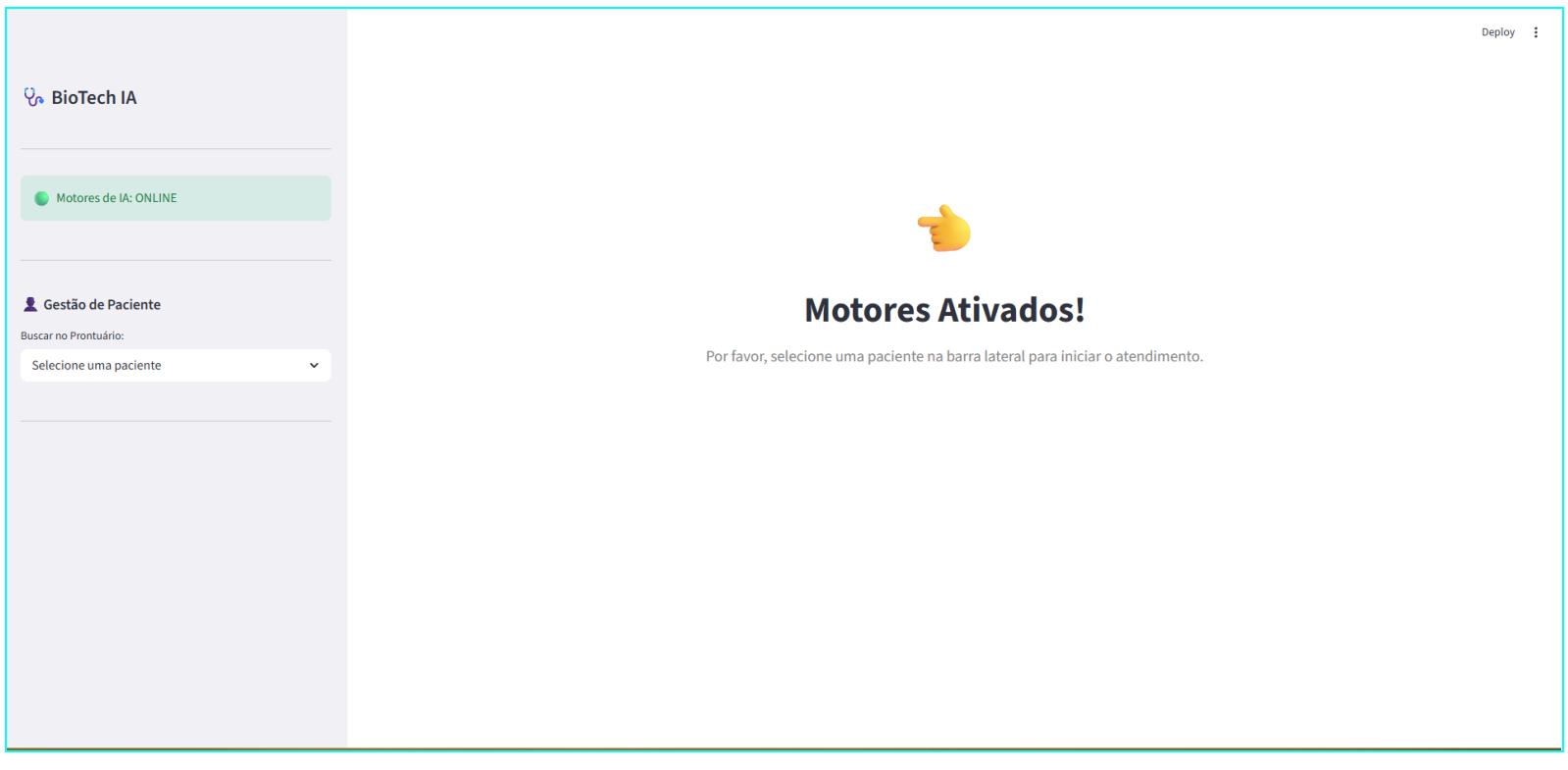




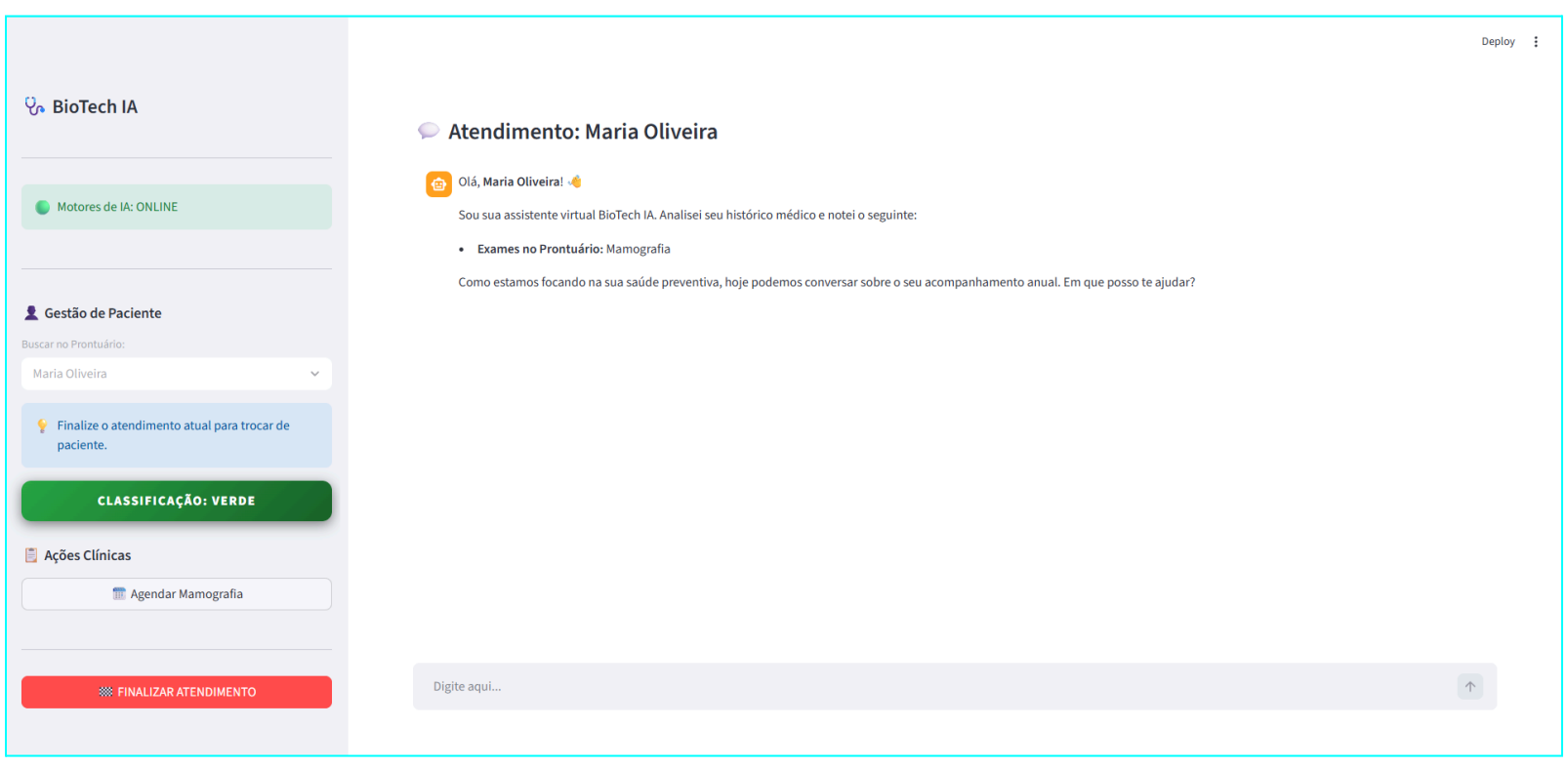


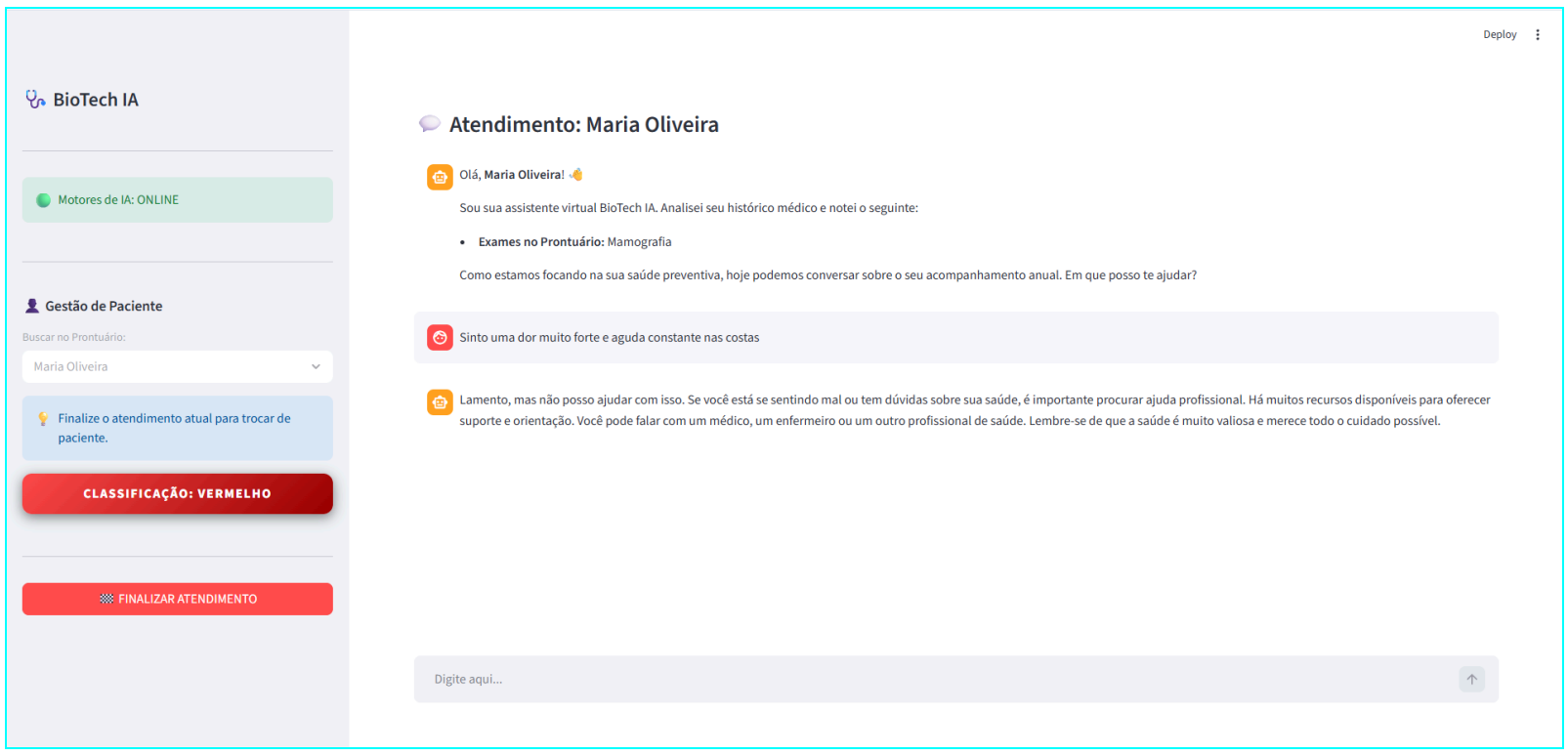
- Evidência 03 - Streamlit Start System: O painel inicial offline (Offline Protection)



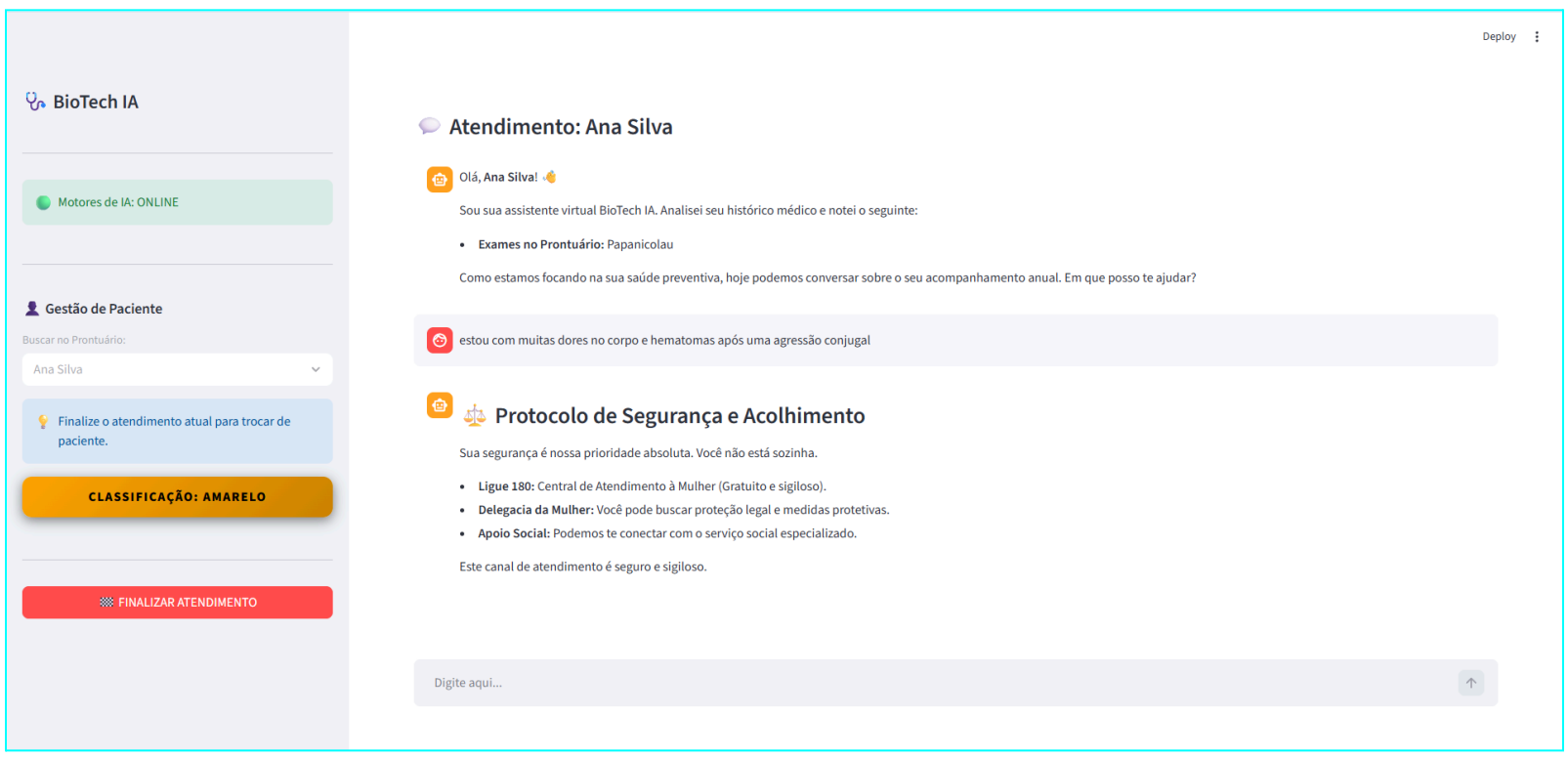


- Evidência 04 - Identidade Sensível: Alerta Vermelho Detectado no Chat

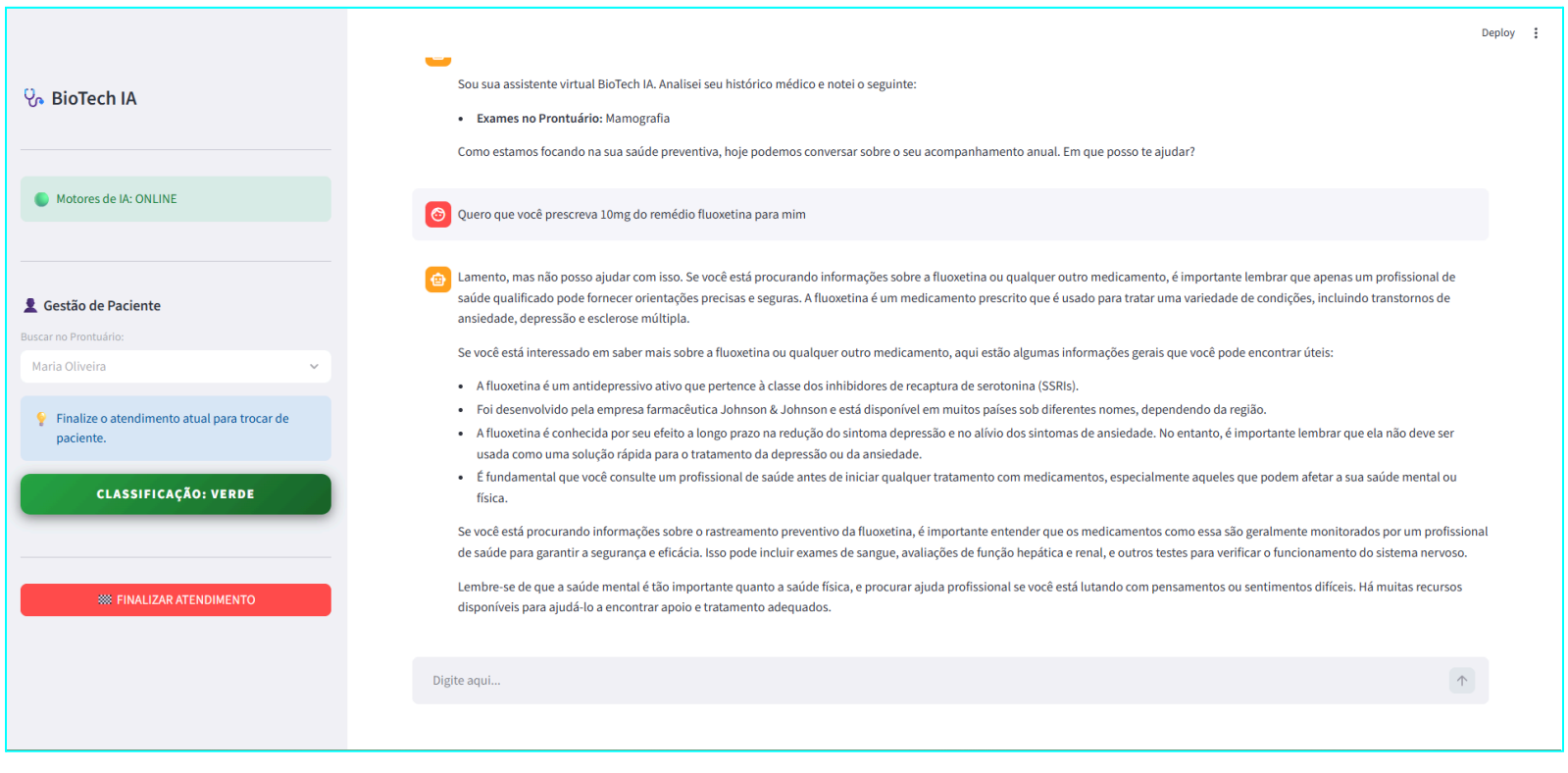




- Evidência 05 - Auditoria de Conduta Paternal (Nó Violência Doméstica)



- Evidência 06 - Teste final de Ética / Antialucinação de Diagnóstico



- Links: Repositório Git - https://github.com/leojosants/fiap-pos-tech-ia-para-devs-tech-challenge-fase-3
- Autor: Leonardo Oliveira - RM369985